

G1 Locomotion

李子墨

学号 241098038

3234454007@qq.com

Abstract

I trained a velocity-tracking policy with PPO for the Unitree G1 humanoid and adapted it to achieve goal-tracking behavior. In addition, I experimented with two out-of-distribution (OOD) scenarios for the policy—OOD speed and OOD terrain—and then fine-tuned the policy to overcome the OOD failures.

1 Introduction

The goal is to make Unitree G1 walk to a target in Isaac Lab. I solve this in two steps.

Training: I use PPO to train a velocity-tracking policy. Its command is (v_x, v_y, ω_z) , where v_x is forward speed, v_y is lateral speed, and ω_z is turning speed.

Inference: I keep the trained policy fixed. A simple proportional controller converts the relative goal position into a velocity command. The policy then follows that command to reach the goal.

I also test the policy in two situations outside its training distribution:

- **Faster commands:** the base policy is trained with $v_x \in [0, 1]$ m/s and tested at speeds up to 2 m/s.
- **Rough terrain:** the flat-ground policy is tested and fine-tuned on stairs, boxes, slopes, and height noise.

In the recorded tests, the base policy falls at $v_x = 1.4$ m/s, while the fine-tuned policy walks at 2 m/s. After a short rough-terrain fine-tune, the policy also walks over the tested uneven surfaces without a height scan.

2 Related Work

Bipedal locomotion. Classical biped controllers often use simplified models such as SLIP [Blickhan, 1989], LIP [Kajita et al., 2001], and ZMP [Vukobratović and Borovac, 2004] to plan stable motion. These methods model balance directly but require carefully designed controllers.

Learning locomotion. Deep reinforcement learning can learn locomotion directly from simulated experience. With thousands of environments running in parallel, a velocity-tracking policy can be trained efficiently [Rudin et al., 2022]. I follow that approach for G1.

Training tools. PPO is an actor–critic algorithm that limits each policy update with a clipped objective [Schulman et al., 2017]. I use PPO from RSL-RL, Isaac Sim for GPU-parallel physics, and Isaac Lab for the G1 robot and locomotion environment.

3 Training

I simulate Unitree G1 in Isaac Lab and train a velocity-tracking policy with PPO (RSL-RL). 4096 environments run in parallel on GPU; each step the policy outputs joint targets, physics advances, and rewards are collected for the update. The policy follows a randomly sampled command $\mathbf{c}_t = (v_x, v_y, \omega_z)$.

3.1 MDP

The learning objective is to make G1 follow a commanded planar velocity $\mathbf{c}_t = (v_x, v_y, \omega_z)$. I use the official flat-ground task `G1FlatEnvCfg` and disable its height scanner. One control step proceeds as follows: the policy observes the robot and the current command, outputs a target offset for every actuated joint, and then the simulator advances the physics and evaluates how well the resulting motion follows the command.

Formally, the simulator state s_t contains the complete physical state of the robot and environment. The policy does not receive s_t directly; it receives the 123-dimensional observation

$$\mathbf{o}_t = \left[\mathbf{v}_t^b, \boldsymbol{\omega}_t^b, \mathbf{g}_t^b, \mathbf{c}_t, \mathbf{q}_t - \mathbf{q}_0, \dot{\mathbf{q}}_t, \mathbf{a}_{t-1} \right], \quad \mathbf{o}_t \in \mathbb{R}^{123}.$$

Here, $\mathbf{v}_t^b$ and $\boldsymbol{\omega}_t^b$ are the base linear and angular velocities (3 values each), $\mathbf{g}_t^b$ is the gravity direction expressed in the base frame (3), and $\mathbf{c}_t$ is the velocity command (3). The remaining terms are the 37 joint positions relative to the default pose, the 37 joint velocities, and the previous 37-dimensional action. These dimensions sum to $3 + 3 + 3 + 3 + 37 + 37 + 37 = 123$. Small uniform noise is added to the body and joint measurements during training. Because the height scanner is disabled, the policy receives no direct description of the terrain.

Action and transition. The action $\mathbf{a}_t \in \mathbb{R}^{37}$ does not specify joint torque directly. Instead, each component is a position offset from the default pose:

$$\mathbf{q}_t^{\text{target}} = \mathbf{q}_0 + 0.5 \mathbf{a}_t.$$

The joint controllers track these targets while Isaac Sim advances the robot dynamics from s_t to s_{t+1} . Including $\mathbf{a}_{t-1}$ in the next observation lets the policy account for its previous target when producing a smooth action sequence.

Reward and termination. The reward is the weighted sum supplied by the official G1 flat task. Its largest positive terms reward tracking the commanded XY velocity and yaw rate (weight +1.0 each). A feet-air-time term (+0.75) encourages stepping. Penalties discourage foot sliding (−0.1), body tilt (−1.0), vertical velocity (−0.2), roll and pitch angular velocity (−0.05), and joint deviation or limit violations (weights from −1.0 to −0.05). Smaller regularizers penalize rapid action changes (-5×10^{-3}), joint torques (-2×10^{-6}), and joint accelerations (-1×10^{-7}). Termination incurs a reward of −200. Thus,

maximizing return asks the robot to follow the command while maintaining a stable, smooth, physically plausible gait.

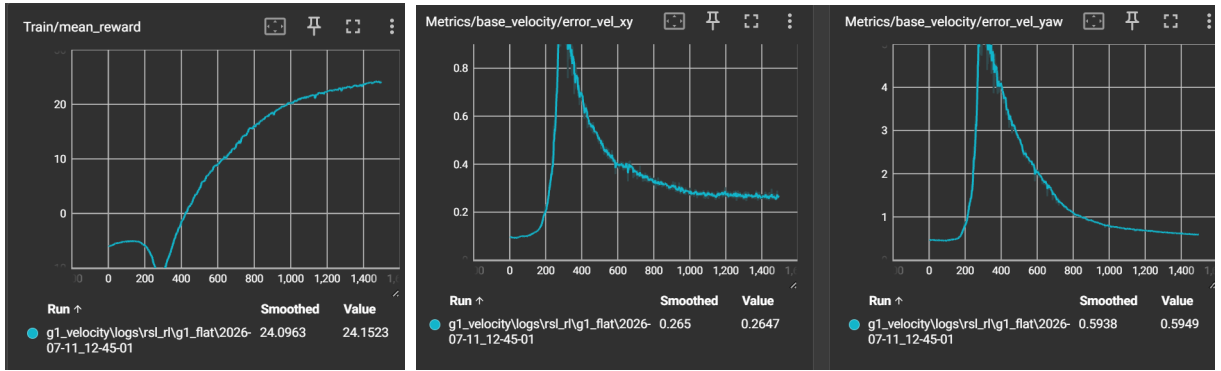
PPO configuration. The actor and critic use MLPs with hidden layers [256, 128, 128]. PPO collects 24 steps from each of 4096 parallel environments per iteration, with clipping parameter 0.2, discount $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, and entropy coefficient 0.008.

3.2 Velocity-policy training

I train one policy to follow randomly sampled velocity commands on flat ground. The command ranges are

$$v_x \in [0, 1] \text{ m/s}, \quad v_y \in [-0.5, 0.5] \text{ m/s}, \quad \omega_z \in [-1, 1] \text{ rad/s},$$

where v_x controls forward motion, v_y controls lateral motion, and ω_z controls turning. A small fraction of environments receive the standing command $\mathbf{c}_t = \mathbf{0}$, so the same policy also learns to remain still. Training runs for 1750 iterations. Figure 1 shows the mean reward rising steadily while the velocity-tracking errors fall, and Figure 2 shows the resulting gait on flat ground.



Mean reward.

Velocity tracking errors.

Figure 1: Base velocity-policy training curves.

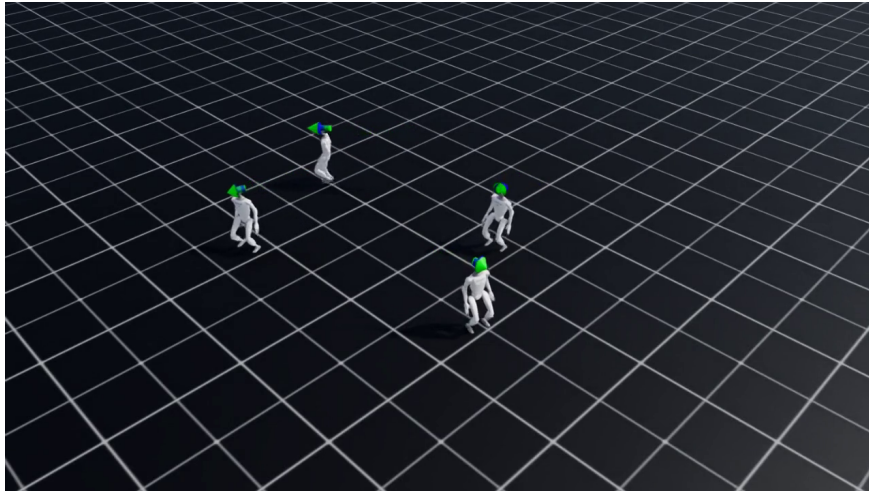


Figure 2: Trained velocity-tracking policy on flat ground (random commands).

4 Inference

At inference, I freeze the policy weights and use a proportional controller to convert the goal displacement into (v_x, v_y, ω_z) . The policy follows these commands with the same observation and action spaces used during training.

Goal-to-velocity controller. At every control step, the world-frame goal is expressed in the robot’s yaw frame as (d_x, d_y) . The angle $\psi = \text{atan2}(d_y, d_x)$ is the direction from the robot to the goal in that frame. The velocity command is then computed as follows:

1. If $\sqrt{d_x^2 + d_y^2} < 0.35$ m, set $\mathbf{c}_t = \mathbf{0}$ so that the robot stands near the goal.
2. Otherwise, apply proportional control:

$$v_x = \text{clip}(k_\ell d_x, 0, 1), \quad v_y = \text{clip}(k_\ell d_y, -0.5, 0.5), \quad \omega_z = \text{clip}(k_a \psi, -1, 1),$$

with $k_\ell = 1.0$ and $k_a = 1.5$.

Clipping keeps every command within the ranges seen during training. In particular, the controller never requests backward motion; when a goal is behind the robot, yaw control turns the robot toward it before substantial forward motion is commanded.

Result. In the recorded simulation, the frozen policy walks toward the marked goal and stands after entering the 0.35 m stopping region (Figure 3).

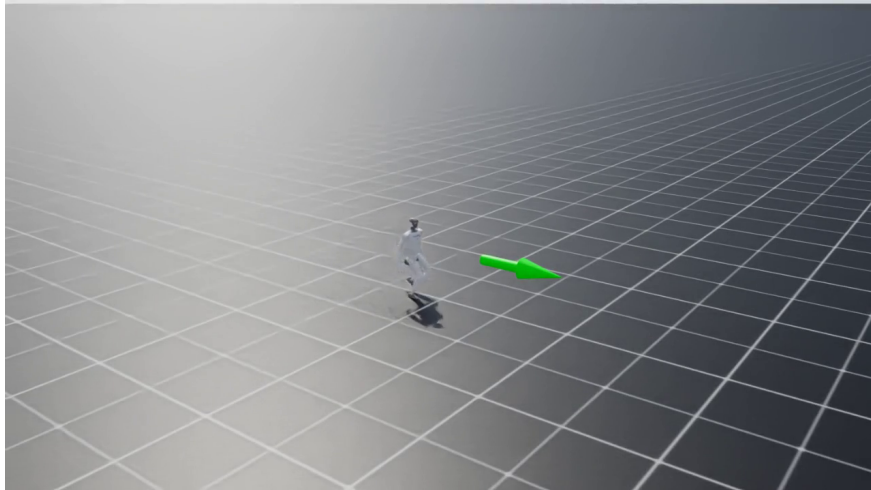


Figure 3: Goal-directed inference. The proportional controller steers the frozen velocity policy toward the green marker.

5 Exploratory Experiment I: Forward-Speed Generalization

The base policy was trained with forward commands no greater than 1 m/s. This experiment asks what happens when it receives a faster command and whether fine-tuning can extend its usable speed range. The robot, terrain, observation, and action remain unchanged; only the magnitude of the forward command moves outside the original training distribution.

5.1 Fine-tuning

The wide- v_x policy is initialized from the base policy and fine-tuned with forward commands up to 2 m/s; the network architecture and goal controller remain unchanged. Its lateral and yaw command ranges are narrowed to emphasize forward locomotion:

Policy	v_x (m/s)	v_y (m/s)	ω_z (rad/s)
Base	[0, 1]	[-0.5, 0.5]	[-1, 1]
Wide- v_x	[0, 2]	[-0.2, 0.2]	[-0.5, 0.5]

Table 1: Command distributions used to train the two policies.

This command distribution focuses the fine-tuning on forward locomotion. Figure 4 shows that the mean reward recovers while the velocity-tracking errors decrease.

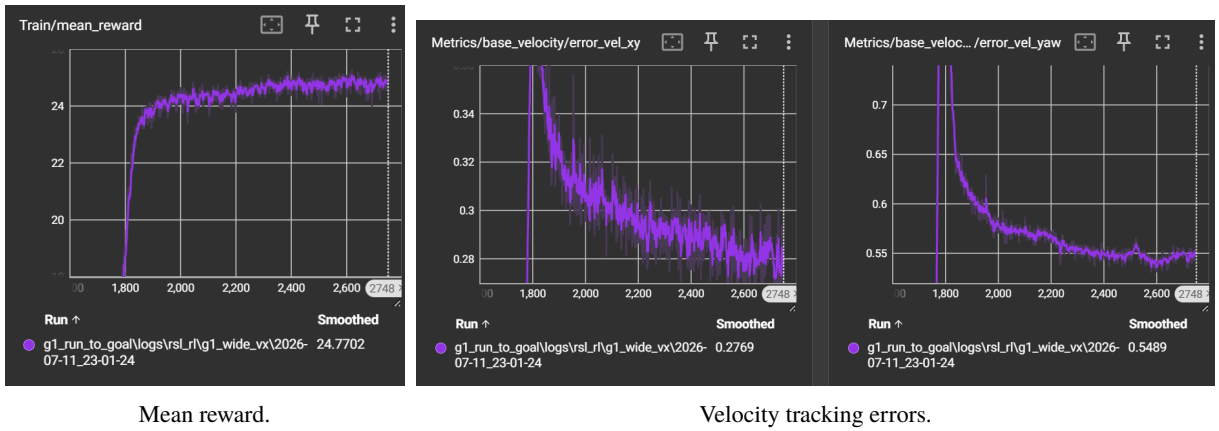


Figure 4: Wide- v_x fine-tuning curves.

5.2 Evaluation protocol

Both policies receive fixed, pure-forward commands $\mathbf{c} = (v_x, 0, 0)$. The sweep covers 0.2 to 2.0 m/s; commands through 1.0 m/s are in-distribution for the base policy, while commands from 1.1 to 2.0 m/s are out-of-distribution. Automatic heading commands and standing environments are disabled so that the commanded forward speed is the only test variable.

I record a simulation clip at each speed and compare whether the robot remains upright and follows the command.

5.3 Observed outcome

In the recorded sweep, the base policy remains upright at $v_x = 1.3$ m/s but falls at 1.4 m/s and at the higher tested commands. After fine-tuning, the wide- v_x policy remains upright and follows the forward command through 2.0 m/s. Fine-tuning therefore extends the tested command range beyond that of the base policy.

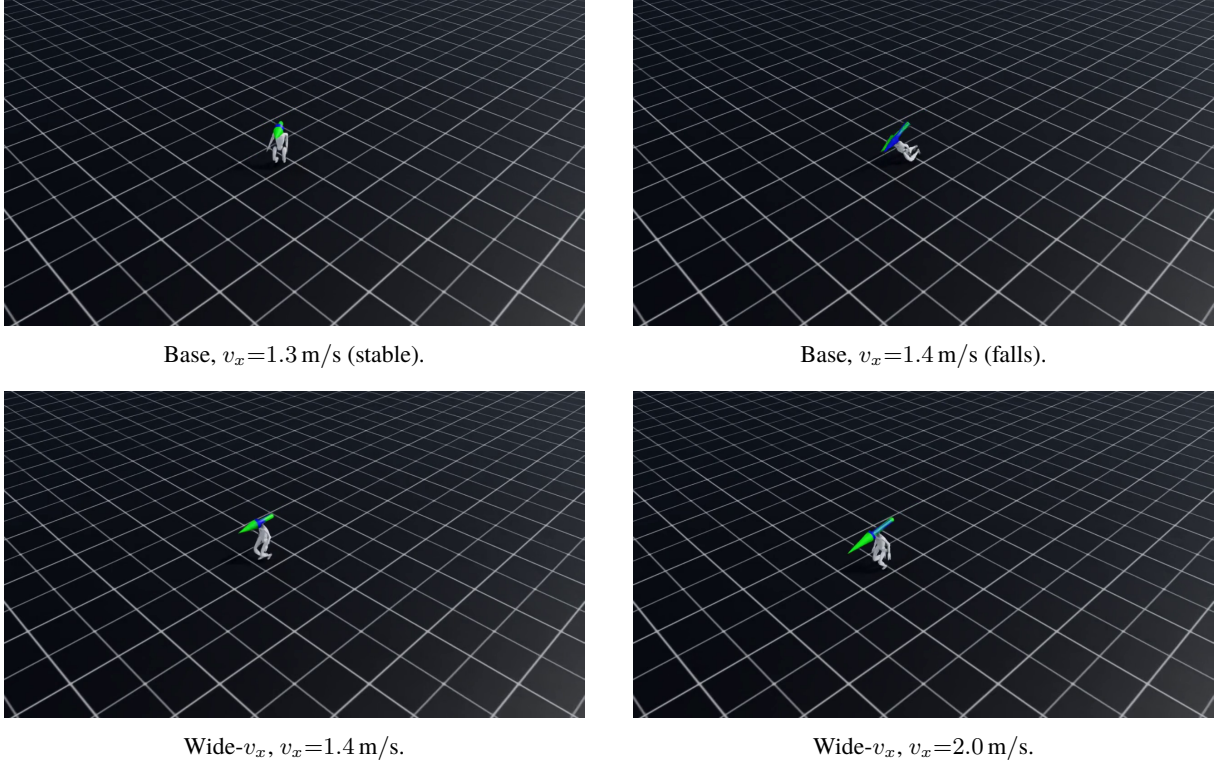


Figure 5: OOD forward-speed frames. The base policy holds at $v_x=1.3$ m/s but collapses at $v_x=1.4$ m/s; wide- v_x recovers at $v_x=1.4$ m/s and still walks at $v_x=2.0$ m/s.

6 Exploratory Experiment II: Rough-Terrain Fine-Tuning

This experiment asks whether the flat G1 velocity policy can transfer to uneven ground **without** changing the observation dimension, so that its pretrained weights remain compatible.

6.1 Motivation

Official rough terrains (stairs, random boxes, height noise, slopes) usually come with a height scanner in the policy observation. The flat policy has no height scan (123-D). Enabling the scanner would change the network input size, so the pretrained checkpoint could no longer be loaded. Can I fine-tune on rough **physics** with the height scan left off and still get usable walking?

6.2 Setup

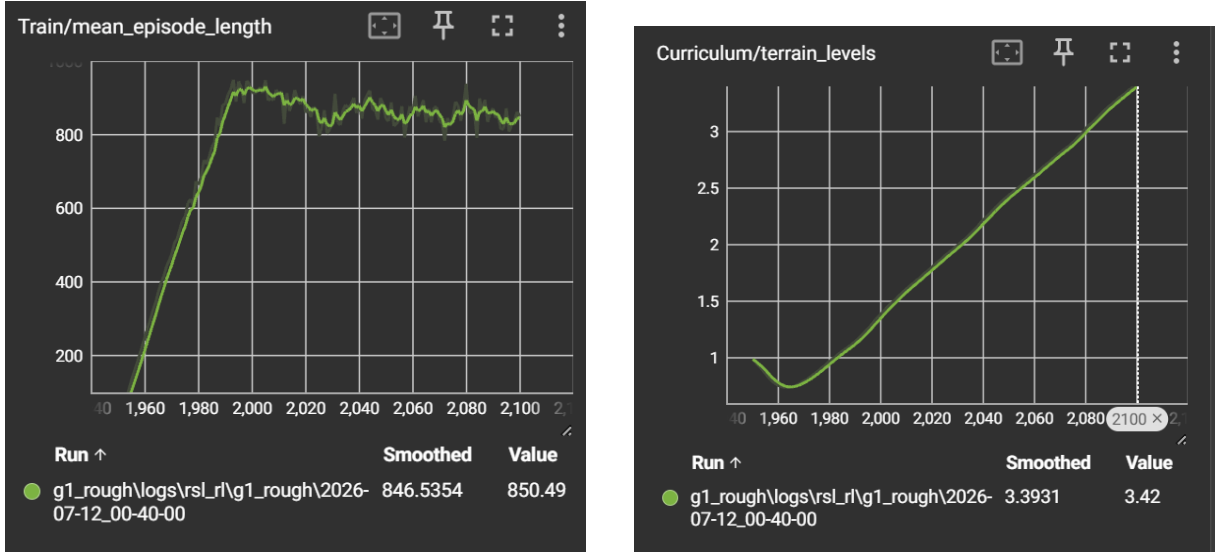
Item	Choice
Terrain	ROUGH_TERRAINS_CFG + curriculum from level 0
Observations	Same as flat: no <code>height_scan</code>
Initialization	Base flat velocity policy (not the wide- v_x policy)
Commands	$v_x \in [0, 2]$, $v_y \in [-0.5, 0.5]$, $\omega_z \in [-1, 1]$
Rewards	Official G1 rough; <code>lin_vel_z_12</code> = 0
Envs	4096

Table 2: Rough-terrain fine-tuning setup.

The fine-tune starts from the base flat policy, so it widens the forward command range to $[0, 2]$ m/s and changes the terrain at the same time. The vertical-velocity penalty `lin_vel_z_12` is set to zero because stepping up and down stairs requires vertical base motion that the flat reward would punish.

6.3 Training

The flat policy is fine-tuned on rough terrain for 351 iterations. Figure 6 shows the mean episode length rising and holding near 850 control steps, while the terrain curriculum climbs from level 1 toward about 3.4.



Mean episode length.

Terrain curriculum level.

Figure 6: Rough-terrain fine-tuning: episode length and curriculum progress.

6.4 Outcome

After fine-tuning, the policy can walk over stairs, random boxes, slopes, and height noise without a height scan. It still falls on some of the harder tiles because it cannot see the terrain ahead and must react after contact. More training would probably help, but the GPU had other appointments on its calendar.

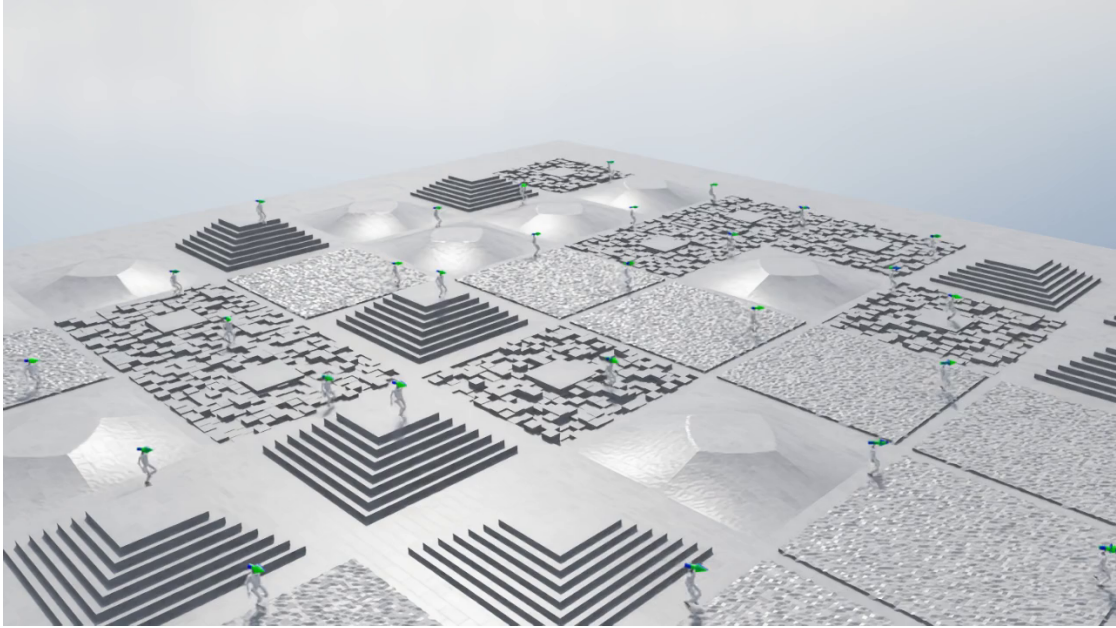


Figure 7: Rough-terrain demo after fine-tuning. The policy navigates stairs, random boxes, height noise, and slopes without a height scan.

7 Summary

The main takeaway is that a velocity policy can be reused for goal tracking: a simple proportional controller generates velocity commands, while the learned policy handles walking. The experiments also show that the training distribution matters. Fine-tuning extends the tested forward speed from 1.3 m/s to 2.0 m/s and allows the flat policy to walk on rough terrain without changing its observation size.

The main limitations are that the speed and rough-terrain results are based on qualitative videos rather than repeated trials or numerical benchmarks. The rough policy is fine-tuned only briefly and has no height scan, so it cannot see upcoming terrain. I evaluate the policy only in simulation.

References

- Reinhard Blickhan. The spring-mass model for running and hopping. *Journal of Biomechanics*, 22 (11-12):1217–1227, 1989.
- Shuuji Kajita, Fumio Kanehiro, Kenji Kaneko, Kazuhito Yokoi, and Hirohisa Hirukawa. The 3d linear inverted pendulum mode: A simple modeling for a biped walking pattern generation. In *Proceedings of the 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems*, volume 1, pages 239–246, 2001.
- Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. *Conference on Robot Learning*, 2022.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Miomir Vukobratović and Branislav Borovac. Zero-moment point—thirty five years of its life. *International Journal of Humanoid Robotics*, 1(01):157–173, 2004.